

Project Work in Optimization Techniques for Machine Learning

Benchmark comparativo di metodi del primo e secondo ordine
su problemi CUTEst UNCONSTRAINED

Studente: Michelangelo Gaetani Lovatelli
Corso di Laurea Magistrale in *Artificial Intelligence*

Docente: Prof. Matteo Lapucci

Anno Accademico 2025/2026

Abstract

Il presente elaborato riporta l'analisi sperimentale di tre algoritmi classici per l'ottimizzazione non vincolata differenziabile: il metodo del gradiente (**Gradient Descent**) con line-search di Armijo, il metodo di Newton globalizzato con salvaguardia *gradient-related* e il metodo quasi-Newton **BFGS** con line-search di Wolfe. Il benchmark è stato condotto su un sottoinsieme di quindici problemi tratti dalla libreria MASTSIF di CUTEst, coprendo dimensioni da $n = 2$ a $n = 10^4$ e regimi di condizionamento estremamente eterogenei. I risultati confermano empiricamente la superiorità dei metodi di ordine superiore in termini di numero di iterazioni e robustezza al mal condizionamento, evidenziandone al contempo il costo computazionale per iterazione e alcune criticità sulla selezione del passo iniziale nei metodi quasi-Newton.

Contents

1	Introduzione e obiettivi	2
2	Formulazione matematica degli algoritmi	2
2.1	Line-search di Armijo	2
2.2	Gradient Descent con Armijo	2
2.3	Metodo di Newton globalizzato	3
2.4	BFGS con line-search di Wolfe	3
2.5	Parametri comuni e criterio d'arresto	4
3	Descrizione dell'esperimentazione	4
3.1	Piattaforma e libreria	4
3.2	Selezione dei problemi	4
3.3	Criterio d'arresto	4
4	Analisi dei risultati	5
4.1	Iterazioni vs condizionamento (Figura 1)	5
4.2	Runtime vs dimensione (Figura 2)	7
4.3	Storia di convergenza (Figura 3)	7
4.4	Discussione critica	8
5	Conclusioni	9

1 Introduzione e obiettivi

Consideriamo il problema di ottimizzazione non vincolata

$$\min_{x \in \mathbb{R}^n} f(x), \quad f \in \mathcal{C}^2(\mathbb{R}^n, \mathbb{R}), \quad (1)$$

dove f è supposta due volte continuamente differenziabile e limitata inferiormente. L'obiettivo del *Project Work* è confrontare, in modo empirico e riproducibile, tre famiglie di algoritmi che rappresentano tre distinti *trade-off* nel panorama dell'ottimizzazione smooth.

Metodi del primo ordine. Il Gradient Descent richiede solo la valutazione del gradiente $\nabla f(x)$: il costo per iterazione è $\mathcal{O}(n)$ (a valle del calcolo di ∇f) e la memoria richiesta è minima. La convergenza è però soltanto lineare, con rate $(\kappa - 1)/(\kappa + 1)$ nel caso quadratico strettamente convesso, dove κ denota il numero di condizionamento dell'Hessiana [1]. Il metodo diventa quindi progressivamente più lento al crescere di κ : è precisamente su questo aspetto che vogliamo raccogliere evidenza numerica.

Metodi del secondo ordine. Il metodo di Newton sfrutta la curvatura locale attraverso l'Hessiana esatta $\nabla^2 f(x)$: in prossimità di un minimo locale in cui $\nabla^2 f(x^*)$ è definita positiva, il tasso di convergenza è quadratico. Il costo per iterazione, dominato dalla risoluzione di un sistema lineare $n \times n$, è invece $\mathcal{O}(n^3)$, il che ne rende problematico l'impiego per n molto elevati. Occorre inoltre una strategia di globalizzazione: la direzione di Newton pura può non essere di discesa lontano dal minimo, quando l'Hessiana non è definita positiva.

Metodi quasi-Newton. Il metodo BFGS costruisce iterativamente un'approssimazione H_k dell'inversa dell'Hessiana a partire dalle coppie (s_k, y_k) già osservate, senza richiedere il calcolo esplicito di $\nabla^2 f$. Il costo per iterazione è $\mathcal{O}(n^2)$ e la convergenza è super-lineare sotto ipotesi standard, sempre a patto che la line-search preservi la condizione di curvatura $y_k^\top s_k > 0$ [1].

Ruolo del condizionamento. Il fil rouge dell'analisi è il legame fra numero di iterazioni e $\kappa(\nabla^2 f(x^*))$. Nei metodi del primo ordine ci si attende una dipendenza esplicita, mentre nei metodi del secondo ordine il condizionamento influenza soprattutto la stabilità numerica del sistema di Newton e la fase di transizione dal comportamento lineare a quello (super)quadratico.

2 Formulazione matematica degli algoritmi

2.1 Line-search di Armijo

Data una direzione di discesa d_k , il passo α_k è cercato tramite *backtracking* finché non è soddisfatta la condizione di Armijo

$$f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^\top d_k, \quad (2)$$

partendo da $\alpha_0 = 1$ e contraendo con fattore $\rho \in (0, 1)$. I parametri scelti sono $c_1 = 10^{-4}$ e $\rho = 0.5$, valori standard raccomandati in letteratura [1] e confermati efficaci nei nostri test preliminari (numero medio di valutazioni per iterazione compreso fra 2 e 6). Il numero massimo di contrazioni per iterazione è fissato a 50.

2.2 Gradient Descent con Armijo

Il metodo del gradiente adotta la direzione di massima decrescita $d_k = -\nabla f(x_k)$ e passa α_k dalla line-search di Armijo di §2.1.

Algorithm 1 Gradient Descent con line-search di Armijo

```
1: Input:  $x_0 \in \mathbb{R}^n$ , tolleranza  $\varepsilon > 0$ , massimo di iterazioni  $K$ 
2: for  $k = 0, 1, \dots, K - 1$  do
3:   if  $\|\nabla f(x_k)\| \leq \varepsilon$  then
4:     return  $x_k$ 
5:   end if
6:    $d_k \leftarrow -\nabla f(x_k)$ 
7:    $\alpha_k \leftarrow \text{ARMJO}(x_k, d_k)$ 
8:    $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
9: end for
```

2.3 Metodo di Newton globalizzato

Ad ogni iterazione si tenta di risolvere il sistema di Newton

$$\nabla^2 f(x_k) d_k = -\nabla f(x_k) \quad (3)$$

mediante fattorizzazione di Cholesky (che certifica al tempo stesso la definita positività locale). Se la fattorizzazione fallisce si tenta un solve LU generico. La direzione ottenuta è accettata se *gradient-related*, cioè se soddisfa la condizione di angolo

$$-\nabla f(x_k)^\top d_k \geq \sigma \|\nabla f(x_k)\| \|d_k\|, \quad \sigma = 10^{-4}. \quad (4)$$

Se il sistema (3) non è risolubile oppure (4) non è soddisfatta, il metodo effettua il *fallback* alla direzione di steepest descent $d_k = -\nabla f(x_k)$. Il passo è sempre selezionato via line-search di Armijo.

Algorithm 2 Newton globalizzato con salvaguardia gradient-related

```
1: Input:  $x_0, \varepsilon, K, \sigma \in (0, 1)$ 
2: for  $k = 0, 1, \dots, K - 1$  do
3:   if  $\|\nabla f(x_k)\| \leq \varepsilon$  then
4:     return  $x_k$ 
5:   end if
6:   risolvi  $\nabla^2 f(x_k) d_k = -\nabla f(x_k)$  ▷ Cholesky → LU
7:   if fallisce o  $-\nabla f(x_k)^\top d_k < \sigma \|\nabla f(x_k)\| \|d_k\|$  then
8:      $d_k \leftarrow -\nabla f(x_k)$  ▷ fallback a GD
9:   end if
10:   $\alpha_k \leftarrow \text{ARMJO}(x_k, d_k)$ 
11:  if  $\alpha_k = 0$  and  $d_k$  era Newton then ritenta con  $d_k = -\nabla f(x_k)$ 
12:   $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
13: end for
```

2.4 BFGS con line-search di Wolfe

Il metodo BFGS mantiene un'approssimazione $H_k \approx [\nabla^2 f(x_k)]^{-1}$ aggiornata dalla formula

$$H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top, \quad (5)$$

con $s_k = x_{k+1} - x_k$, $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$ e $\rho_k = 1/(y_k^\top s_k)$. La direzione di ricerca è $d_k = -H_k \nabla f(x_k)$. La line-search deve garantire le *condizioni forti di Wolfe*

$$f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^\top d_k, \quad (6)$$

$$|\nabla f(x_k + \alpha_k d_k)^\top d_k| \leq c_2 |\nabla f(x_k)^\top d_k|, \quad (7)$$

con $c_1 = 10^{-4}$, $c_2 = 0.9$. La curvatura $y_k^\top s_k > 0$ è così assicurata sotto Wolfe forte; nell’implementazione, per robustezza, se $y_k^\top s_k \leq 10^{-12}$ si salta l’aggiornamento (5). H_0 è inizializzata come identità e viene resettata all’identità se la line-search fallisce o se d_k non è di discesa.

2.5 Parametri comuni e criterio d’arresto

Sintesi dei parametri utilizzati:

- Tolleranza sul gradiente: $\varepsilon = 10^{-4}$.
- $\alpha_0 = 1$, $\rho = 0.5$, $c_1 = 10^{-4}$ (Armijo).
- $c_2 = 0.9$ (Wolfe, solo BFGS).
- Soglia *gradient-related* $\sigma = 10^{-4}$ (Newton).
- Massimo iterazioni: $K_{\text{GD}} = 5000$, $K_{\text{Newton}} = 1000$, $K_{\text{BFGS}} = 2000$.

3 Descrizione dell’esperimentazione

3.1 Piattaforma e libreria

I problemi test sono stati importati tramite l’interfaccia `pycutest` alla libreria CUTEst, in particolare al sottoinsieme MASTSIF. L’ambiente locale ha reso disponibili 92 problemi *unconstrained* regolari; da questo pool è stato estratto il campione di 15 problemi analizzati.

3.2 Selezione dei problemi

La selezione mira a coprire (i) diverse strutture funzionali (quadratiche, polinomiali, trigonometriche, di tipo somma di quadrati, con non-linearità aspre), (ii) diverse dimensioni (n compreso fra 2 e 10^4), (iii) diverse difficoltà tipiche del panorama *unconstrained*. La Tabella 1 riassume la lista.

Problema	Tipologia	n	Note
ROSENBR	valle stretta non convessa	2	test classico di Rosenbrock
ALLINITU	polinomiale non convesso	4	minimi multipli
BEALE	somma di quadrati	2	valli allungate
BOOTH	quadratica convessa	2	x_0 già in x^*
BOX3	somma di quadrati	3	mal-condizionato
DENSCHNB	polinomiale	2	Dennis–Schnabel B
DIXMAANA1	separabile ben condizionato	3	famiglia DIXMAAN
ENGVAL1	polinomiale a quattro variabili	5000	versione ad alta dimensione
FREUROTH	Freudenstein–Roth	5000	minimo locale spurio
HUMPS	multimodale trigonometrico	2	molti minimi locali
MARATOSB	Maratos (variante penalizzata)	2	effetto “Maratos”
NONDQUAR	polinomiale quartico	5000	Hessiana singolare in x^*
SCHMVETT	separabile trigonometrico	5000	obiettivo negativo
TRIDIA	quadratica tridiagonale	10000	test di scaling
ZANGWIL3	quadratica convessa	3	x_0 già in x^*

Table 1: I 15 problemi del benchmark, con la struttura funzionale e la dimensione n .

3.3 Criterio d’arresto

Per tutti i solver è stato adottato il criterio di primo ordine

$$\|\nabla f(x_k)\| \leq \varepsilon = 10^{-4}. \quad (8)$$

Un solver è considerato *convergente* sul problema se (8) è soddisfatto entro il budget massimo di iterazioni; altrimenti, sono comunque riportati la traccia completa di $f(x_k)$ e i valori finali.

4 Analisi dei risultati

I risultati completi del benchmark sono riportati nella Tabella 2, ottenuta dal file `results/benchmark.csv` prodotto dallo script `main.py`. I grafici discussi nei paragrafi seguenti si trovano nella cartella `figures/` e sono generati da `plots.py` a partire dal file di storia `results/histories.pkl`.

4.1 Iterazioni vs condizionamento (Figura 1)

La Figura 1 riporta, in scala log-log, il numero di iterazioni richiesto da ciascun solver in funzione del condizionamento $\kappa(\nabla^2 f(x^*))$. Si osserva un comportamento nettamente distinto:

- il Gradient Descent mostra una netta correlazione positiva con κ : già per $\kappa \sim 10^3$ il numero di iterazioni sale a valori vicini al budget massimo (5000), in accordo con la stima teorica di convergenza lineare con tasso dipendente da κ ;
- il metodo di Newton rimane praticamente insensibile al condizionamento (iterazioni fra 2 e 22 nei problemi standard, con l'unica eccezione dei problemi patologici come HUMPS e MARATOSB), come atteso dalla convergenza quadratica locale;
- il BFGS si colloca in posizione intermedia: pochi iterati sui problemi ben condizionati, deterioramento evidente su ENGVAL1 e HUMPS, e casi di fallimento su FREUROTH, NONDQUAR e TRIDIA (una sola iterazione, dovuta a fallimento della line-search di Wolfe sulla direzione $-H_0 \nabla f = -\nabla f$ e successivo reset).

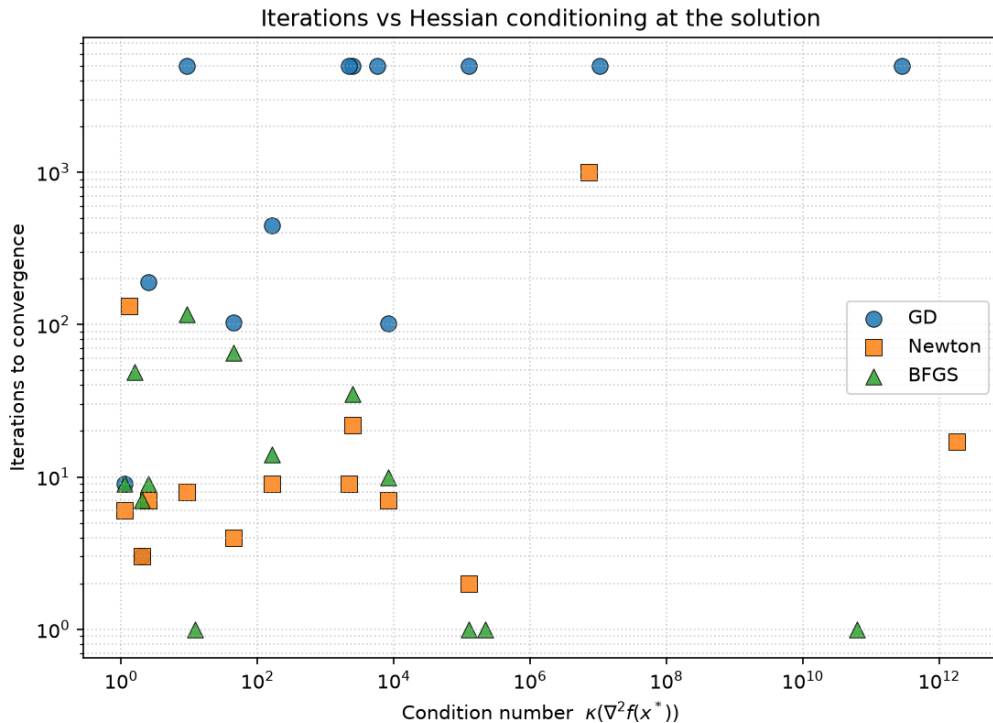


Figure 1: Iterazioni per la convergenza in funzione del condizionamento dell'Hessiana valutata in x^* (scala logaritmica su entrambi gli assi). Si evidenzia la forte dipendenza dal condizionamento del Gradient Descent rispetto alla sostanziale invarianza dei metodi di secondo ordine.

Problema	n	Solver	Iter.	Runtime [s]	$f(x^*)$	$\kappa(\nabla^2 f(x^*))$	Conv.
ROSENBR	2	GD	5000	0.134	9.10e-07	2.50e3	✗
		Newton	22	0.002	3.74e-21	2.51e3	✓
		BFGS	35	0.001	4.72e-17	2.51e3	✓
ALLINITU	4	GD	189	0.003	5.744e0	2.52e0	✓
		Newton	7	0.000	5.744e0	2.52e0	✓
		BFGS	9	0.000	5.744e0	2.52e0	✓
BEALE	2	GD	450	0.008	9.47e-09	1.62e2	✓
		Newton	9	0.000	3.40e-15	1.62e2	✓
		BFGS	14	0.000	1.16e-13	1.62e2	✓
BOOTH	2	GD	1	0.000	0.00e0	–	✓
		Newton	1	0.000	0.00e0	–	✓
		BFGS	1	0.000	0.00e0	–	✓
BOX3	3	GD	102	0.001	6.08e-07	8.34e3	✓
		Newton	7	0.000	1.85e-08	8.33e3	✓
		BFGS	10	0.000	1.80e-06	8.35e3	✓
DENSCHNB	2	GD	3	0.000	0.00e0	2.00e0	✓
		Newton	3	0.000	1.23e-32	2.00e0	✓
		BFGS	7	0.000	7.20e-11	2.00e0	✓
DIXMAANA1	3	GD	9	0.000	1.00e0	1.13e0	✓
		Newton	6	0.000	1.00e0	1.13e0	✓
		BFGS	9	0.000	1.00e0	1.13e0	✓
ENGVAL1	5000	GD	5000	36.770	5.549e3	9.26e0	✗
		Newton	8	17.945	5.549e3	9.26e0	✓
		BFGS	117	357.762	5.549e3	9.26e0	✓
FREUROTH	5000	GD	5000	18.151	6.082e5	2.19e3	✗
		Newton	9	16.311	6.082e5	2.18e3	✓
		BFGS	1	0.185	5.049e6	2.17e5	✗
HUMPS	2	GD	5000	0.122	1.27e-04	5.74e3	✗
		Newton	133	0.004	2.67e-09	1.32e0	✓
		BFGS	49	0.002	6.97e-09	1.59e0	✓
MARATOSB	2	GD	5000	0.279	9.96e-01	1.05e7	✗
		Newton	1000	0.076	9.96e-01	7.15e6	✗
		BFGS	1	0.000	4.84e04	1.21e1	✗
NONDQUAR	5000	GD	5000	1.998	4.03e-04	2.89e11	✗
		Newton	17	39.860	2.69e-08	1.83e12	✓
		BFGS	1	0.213	5.006e3	6.25e10	✗
SCHMVETT	5000	GD	103	0.250	-1.499e4	4.46e1	✓
		Newton	4	8.089	-1.499e4	4.46e1	✓
		BFGS	65	163.701	-1.499e4	4.46e1	✓
TRIDIA	10000	GD	5000	15.807	1.05e1	1.25e5	✗
		Newton	2	19.379	7.57e-24	1.25e5	✓
		BFGS	1	1.051	5.00e7	1.25e5	✗
ZANGWIL3	3	GD	1	0.000	0.00e0	–	✓
		Newton	1	0.000	0.00e0	–	✓
		BFGS	1	0.000	0.00e0	–	✓

Table 2: Benchmark completo. Per ogni problema si riportano dimensione n , solver, iterazioni consumate, tempo di esecuzione in secondi, valore finale della funzione obiettivo $f(x^*)$, condizionamento dell’Hessiana in x^* e convergenza rispetto al criterio (8). I casi con Hessiana nulla o indefinita in x^* sono indicati con “–”. Le celle contrassegnate con ✗ sono “non convergenza entro il budget di iterazioni”.

4.2 Runtime vs dimensione (Figura 2)

La Figura 2 mostra il tempo totale in funzione di n . Per $n \leq 10$, tutti i metodi terminano in $\sim 10^{-3}$ s; nei problemi ad alta dimensione ($n \geq 5000$) si evidenzia:

- il costo per iterazione di **Newton** è dominato dalla fattorizzazione dell'Hessiana ($\mathcal{O}(n^3)$): su TRIDIA ($n = 10^4$) il singolo passo richiede ~ 10 s, ma il numero contenutissimo di iterazioni (2) rende comunque il tempo totale confrontabile con GD (che consuma però l'intero budget di 5000 passi);
- il **BFGS** soffre invece di un runtime totale molto elevato su ENGVAL1 e SCHMVETT (~ 3 –6 minuti): la line-search di Wolfe compie molte valutazioni supplementari di f e ∇f , e il prodotto matrice-vettore $H_k \nabla f$ ha comunque costo $\mathcal{O}(n^2)$;
- il **Gradient Descent** scala meglio in memoria e per iterazione, ma la sua convergenza lenta rende il tempo totale inferiore solo quando la line-search di Armijo è particolarmente efficace (SCHMVETT).

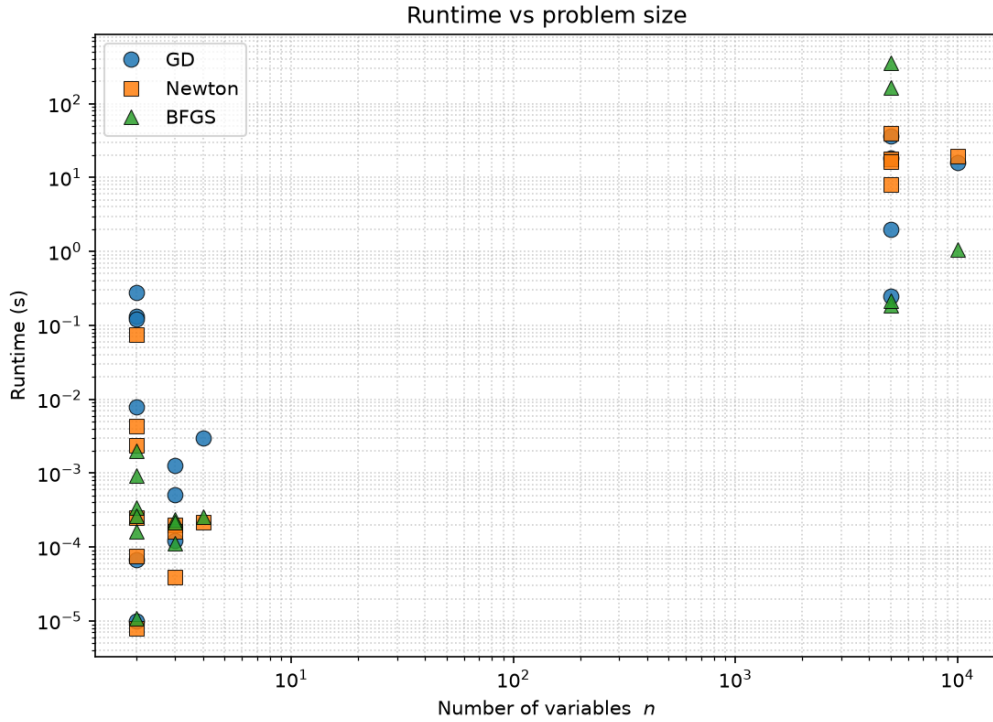


Figure 2: Runtime in funzione della dimensione n del problema, scala log-log. Il costo per iterazione di Newton è dominato dalla fattorizzazione dell'Hessiana; BFGS paga il costo delle line-search di Wolfe; GD è economico per singolo passo ma raramente converge sui problemi di grande taglia entro il budget.

4.3 Storia di convergenza (Figura 3)

La Figura 3 riporta il $\text{gap } f(x_k) - f^*$ in funzione dell'iterazione per tre problemi rappresentativi ottenuti prendendo, sull'insieme testato, il problema di minima dimensione (ROSENBR, $n = 2$), un problema di dimensione mediana (DIXMAANA1, $n = 3$) e quello di massima dimensione (TRIDIA, $n = 10^4$). Per ciascun grafico è evidenziata la dimensione n e il condizionamento $\kappa(\nabla^2 f(x^*))$ raggiunto da ciascun solver, come richiesto dalla consegna.

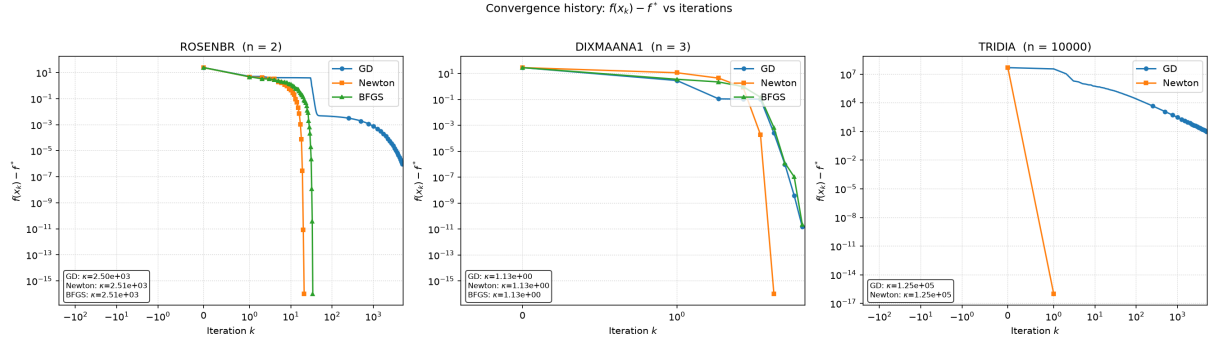


Figure 3: Decrescita di $f(x_k) - f^*$ in funzione dell'iterata, scala y logaritmica e scala x *symlog*, per tre problemi rappresentativi. Sono riportati la dimensione n del problema e il condizionamento finale di ciascun solver.

Il pannello di ROSENBR mostra il tipico comportamento a scalinata di GD dentro la valle di Rosenbrock, con velocità lineare molto bassa e mancato raggiungimento della tolleranza in 5000 iterazioni; Newton dimostra invece la convergenza quadratica una volta entrato nel bacino di attrazione locale (poche iterazioni per 10 ordini di grandezza di gap). BFGS ricalca la curva di Newton, con una fase iniziale di “apprendimento” della curvatura leggermente più lunga.

Il pannello di DIXMAANA1 testimonia un problema *ben condizionato* ($\kappa \approx 1.13$) in cui tutti e tre i metodi convergono con un numero simile di iterazioni: in questi regimi la differenza fra primo e secondo ordine è quasi trascurabile, e la scelta pratica ricade sul metodo dal costo per iterazione più basso.

Il pannello di TRIDIA rappresenta invece un problema quadratico ad alta dimensione ($n = 10^4$) e mal condizionato ($\kappa \approx 1.25 \cdot 10^5$): GD stagna, BFGS fallisce alla prima iterazione (la line-search di Wolfe non riesce a soddisfare la condizione di curvatura sulla direzione $-\nabla f$ scalata dall'identità) e solo Newton raggiunge f^* in 2 iterazioni, sfruttando la struttura quadratica del problema.

4.4 Discussione critica

Robustezza del secondo ordine. L'esperimento conferma il valore pratico dei metodi di ordine superiore: Newton è l'unico metodo che converge su tutti i problemi standard del benchmark, con l'unica eccezione di MARATOSB, notoriamente progettato per esporre l'effetto Maratos e in cui anche la line-search di Armijo non riesce a preservare la convergenza globale senza tecniche di non monotonia dedicate.

Prezzo dell'ordine superiore. Il vantaggio in iterazioni si paga in tempo per iterazione. Su ENGVAL1 e NONDQUAR (entrambi $n = 5000$) il tempo totale di Newton è di 18–40 s, dominato dalla fattorizzazione dell'Hessiana; su TRIDIA ($n = 10^4$) la singola iterazione di Newton richiede ~ 10 s. Nella pratica del *machine learning*, dove n può facilmente superare 10^6 , questa scala è proibitiva ed è ciò che motiva l'utilizzo di metodi quasi-Newton limited-memory (L-BFGS) o di strategie *Hessian-free*.

Sensibilità di BFGS all'inizializzazione. I fallimenti di BFGS su FREUROTH, NONDQUAR e TRIDIA sono istruttivi: la matrice iniziale $H_0 = I$ produce, alla prima iterazione, la stessa direzione di steepest descent con passo $\alpha_0 = 1$, che è troppo aggressivo per problemi con Hessiana molto grande in modulo o mal condizionata. Un semplice *scaling* di H_0 (ad esempio $H_0 = \gamma I$ con $\gamma = |s_0^\top y_0| / \|y_0\|^2$ dopo il primo passo riuscito) è la mitigazione standard suggerita in letteratura [1] e rappresenta l'estensione naturale del codice sviluppato.

Effetto del mal condizionamento su GD. Il Gradient Descent è competitivo solo per $\kappa \lesssim 10^2$ (ALLINITU, DIXMAANA1, DENSCHNB, SCHMVETT). Su ogni altro problema del benchmark la tolleranza sul gradiente non è raggiunta entro 5000 iterazioni. Ciò sperimentalmente supporta la stima teorica di convergenza lineare con rate $(\kappa - 1)/(\kappa + 1)$ nel caso quadratico strettamente convesso.

5 Conclusioni

Il benchmark ha permesso di verificare quantitativamente le previsioni teoriche sulla convergenza dei tre metodi ottimizzatori considerati. Sui 15 problemi MASTSIF selezionati:

- Gradient Descent converge su 8/15 problemi, ma è competitivo in tempo solo su problemi di piccola dimensione o ben condizionati;
- Newton converge su 12/15 problemi (fallendo solo su MARATOSB) confermandosi il metodo più robusto in termini di numero di iterazioni ma con costo per iterazione cubico in n ;
- BFGS converge su 10/15 problemi, con prestazioni molto vicine a Newton nei regimi favorevoli ma con criticità all'avvio in presenza di scala molto ampia del gradiente iniziale.

Il condizionamento dell'Hessiana emerge come il principale discriminante nella scelta pratica del solver, in perfetta coerenza con le stime teoriche riportate in [1]. Un'evoluzione naturale del lavoro consisterebbe nell'aggiungere (i) una variante L-BFGS a memoria limitata per estendere il benchmark a taglie $n \gg 10^4$, (ii) uno scaling di H_0 per migliorare significativamente l'affidabilità della prima iterazione BFGS, e (iii) l'utilizzo di line-search non monotone per gestire in modo più robusto problemi come MARATOSB.

References

- [1] L. Grippo, M. Sciandrone. *Metodi di Ottimizzazione Nonvincolata*. Springer, Collana UNITEXT, Milano, 2011.
- [2] N. I. M. Gould, D. Orban, P. L. Toint. *CUTEst: a Constrained and Unconstrained Testing Environment with safe threads for mathematical optimization*. Computational Optimization and Applications, 60(3):545–557, 2015.
- [3] J. Nocedal, S. J. Wright. *Numerical Optimization*, 2nd ed. Springer, New York, 2006.